Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Операционные системы

Лабораторная работа №11

Выполнили: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2023

Оглавление

Ход работы: 4

Вывод: 8

Листинг: 9

end_buff.c 9

circular_buff.c 10

Задание: реализуйте алгоритм “производитель-потребитель” для конечного буфера и циклического буфера.

Цель: получение навыков синхронизации с использованием мьютексов и семафоров.

Ход работы:

Конечный буфер:

Конечный буфер — это структура данных фиксированного размера, в которую производители могут помещать данные, а потребители могут их извлекать. Однако, если буфер заполнен, производитель должен ждать, пока место освободится, чтобы положить новые данные. Аналогично, если буфер пуст, потребитель должен ждать, пока появятся новые данные для извлечения.

Циклический буфер:

Циклический буфер также фиксированного размера, но он образует кольцо, где данные добавляются или извлекаются по кругу. Когда производитель достигает конца буфера, он начинает с начала, и то же самое происходит с потребителем: когда он достигает конца, он начинает с начала. Это позволяет использовать буфер повторно, избегая блокировки производителей и потребителей, пока в буфере есть свободное место или данные для извлечения.

Создадим две программы для демонстрации синхронизации между производителем и потребителем с использованием условных переменных для безопасного доступа к общему буферу.

Для конечного буфера (end_buff.c – код представлен в листинге)**:**

1. **Производитель** (функция **producer**) постоянно генерирует элементы и пытается поместить их в буфер.
2. Если буфер заполнен (**counter == BUFFER_SIZE**), производитель ждет сигнала от потребителя (**cond_producer**).
3. Как только производитель помещает элемент в буфер, он увеличивает **counter** и отправляет сигнал потребителю (**cond_consumer**).
4. **Потребитель** (функция **consumer**) ждет элементы в буфере.
5. Если буфер пуст (**counter == 0**), потребитель ждет сигнала от производителя (**cond_consumer**).

6. Потребитель забирает элемент из буфера, уменьшает **counter** и отправляет сигнал производителю (**cond_producer**).

Для циклического буфера (circular_buff.c – код представлен в листинге)**:**

1. **Производитель** постоянно генерирует элементы и пытается поместить их в буфер.
2. Если буфер заполнен (**count == BUFFER_SIZE**), производитель ждет сигнала от потребителя (**cond_producer**).
3. Как только производитель помещает элемент в буфер, он изменяет индекс **in** для указания на следующую позицию в циклическом буфере и увеличивает **count**.
4. Производитель отправляет сигнал потребителю (**cond_consumer**).
5. **Потребитель** ждет элементы в буфере.
6. Если буфер пуст (**count == 0**), потребитель ждет сигнала от производителя (**cond_consumer**).
7. Потребитель забирает элемент из буфера, изменяет индекс **out** для указания на следующий элемент для чтения и уменьшает **count**.
8. Потребитель отправляет сигнал производителю (**cond_producer**).

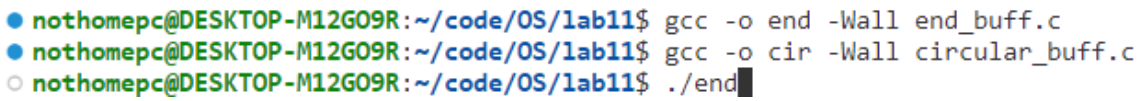


Рис. 1 - Компиляция и запуск программ.



Рис. 2 – Результат работы программы end_buff.c

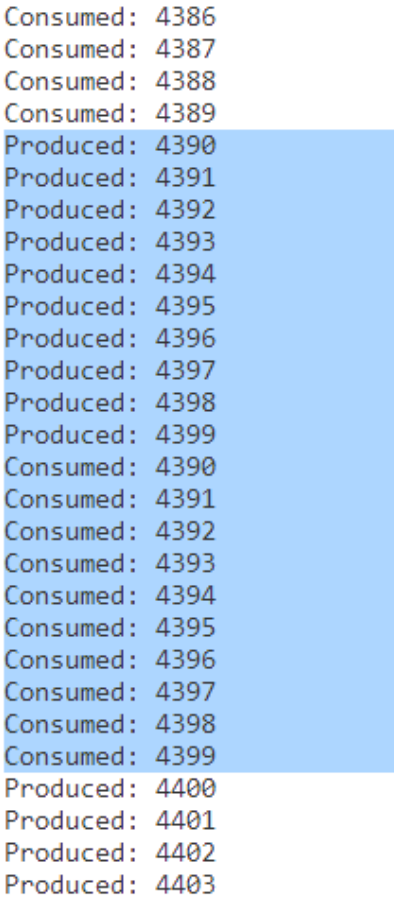


Рис. 2 – Результат работы программы circular_buff.c

Как можно заметить по результатам **конечный** буфер работает по принципу стека (последним пришёл — первым вышел), а **циклический** буфер по принципу очереди (first in, first out «первым пришёл — первым ушёл»).

Вывод:

Мы реализовали алгоритм “производитель-потребитель” для конечного буфера и циклического буферов.

Получили навыки синхронизации с использованием мьютексов и семафоров.

Листинг:

end_buff.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int counter = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond_producer = PTHREAD_COND_INITIALIZER;
pthread_cond_t cond_consumer = PTHREAD_COND_INITIALIZER;

void* producer(void* arg) {
    int item = 0;
    while (1) {
        pthread_mutex_lock(&mutex);
        if (counter == BUFFER_SIZE) {
            pthread_cond_wait(&cond_producer, &mutex);
        }
        buffer[counter++] = item;
        printf("Produced: %d\n", item++);
        pthread_cond_signal(&cond_consumer);
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        if (counter == 0) {
            pthread_cond_wait(&cond_consumer, &mutex);
        }
        printf("Consumed: %d\n", buffer[--counter]);
        pthread_cond_signal(&cond_producer);
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main() {
    pthread_t producer_thread, consumer_thread;
    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);
```

```
    return 0;
}
```

circular_buff.c

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
int count = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond_producer = PTHREAD_COND_INITIALIZER;
pthread_cond_t cond_consumer = PTHREAD_COND_INITIALIZER;

void* producer(void* arg) {
    int item = 0;
    while (1) {
        pthread_mutex_lock(&mutex);
        if (count == BUFFER_SIZE) {
            pthread_cond_wait(&cond_producer, &mutex);
        }
        buffer[in] = item;
        printf("Produced: %d\n", item++);
        in = (in + 1) % BUFFER_SIZE;
        count++;
        pthread_cond_signal(&cond_consumer);
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        if (count == 0) {
            pthread_cond_wait(&cond_consumer, &mutex);
        }
        int consumed = buffer[out];
        printf("Consumed: %d\n", consumed);
        out = (out + 1) % BUFFER_SIZE;
        count--;
        pthread_cond_signal(&cond_producer);
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
```

```
}

int main() {
    pthread_t producer_thread, consumer_thread;
    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    return 0;
}
```